

KijaniKiosk DevOps Foundation

Track A: Infrastructure-First

DevOps Capstone Project

Sharon Ombonya

21 August 2026

Problem

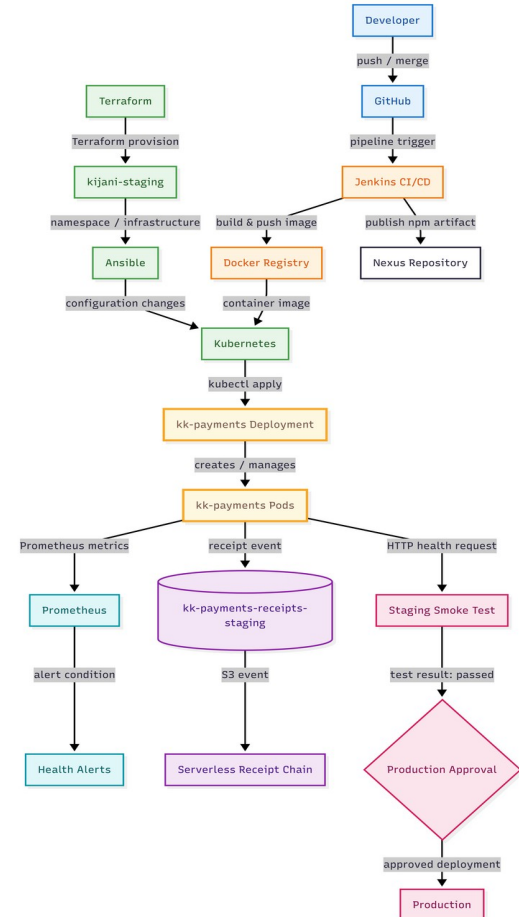
KijaniKiosk initially lacked a repeatable and controlled way to build, deploy, verify, and monitor the kk-payments service. The project needed automated CI/CD, infrastructure provisioning, configuration separation, a Kubernetes staging environment, health monitoring, and a controlled path toward production. The capstone addresses this operational gap by combining Jenkins, Docker, Terraform, Ansible, Kubernetes, Prometheus, and the Week 10 receipt-processing workflow into one documented delivery system.

Architecture

GitHub → Jenkins → Build/Test → Docker → Kubernetes → Monitoring

And:

kk-payments → Receipt Processing



Key Technical Decision

Staging deployment before production approval

I chose:

- A Jenkins pipeline that deploys to kijani-staging, runs a smoke test, and only then presents the production approval gate.

I did not choose:

- Direct automatic deployment from the build stage to production.

Why:

- The staging environment provides a verification point before production and allows a human to review the result before promotion.

Trade-off:

- This improves deployment control and reduces the risk of promoting a failed build, but it adds deployment time and requires staging infrastructure plus human approval.

Production change

- In a larger production environment:
- I would keep the approval control for high-risk changes while using automated tests and progressive delivery to reduce unnecessary waiting.

AI Tooling & Governance

How I used AI

- Used ChatGPT and Google Gemini to troubleshoot Jenkins, Docker, Kubernetes/Minikube, and deployment issues.
- AI suggestions were tested against the actual project environment before implementation.

What AI got right

- Identified that Jenkins needed access to Docker, Kubernetes, and Minikube resources.
- Correctly identified the kubeconfig/Minikube path mismatch inside the Jenkins container.
- Helped identify possible causes of the Jenkins production-stage issue.

What AI got wrong

- ChatGPT initially suggested increasing the overall Jenkins pipeline timeout from 15 → 30 minutes, then suggested increasing it further.
- This did not solve the problem because Jenkins was intentionally waiting for human approval.
- Gemini also offered increasing the timeout as Option 1, which I rejected.

What I changed

- I tested the timeout recommendation and rejected further increases.
- I chose Gemini's Option 2: inspect the running Jenkins pipeline.
- I found the pending approval request, entered the approval reason, and allowed the pipeline to continue.
- This reinforced that AI recommendations are advisory, not authoritative.

Production Gaps & Required Changes

1. Kubernetes / Infrastructure

- Gap: Local Minikube and development infrastructure.
- Change: Move to managed/hardened Kubernetes with stronger security, availability, networking, backups and disaster recovery.

2. Jenkins / Webhook

- Gap: Local Jenkins and temporary ngrok webhook.
- Change: Use production-grade CI/CD and a stable, secured webhook endpoint.

3. Secrets / Monitoring / Receipts

- Gap: Development-level secrets, monitoring and receipt processing.
- Change: Add dedicated secrets management, production monitoring, and stronger reliability, security and error handling.